

RAUL LAB · DOCUMENTACIÓN CANÓNICA DERIVADA DE MKDOCS

Replicant Lab

Infraestructura, hosts, red, despliegues y operación.

Versión reproducible

sha256:74c870a9cbe97df3313f0b24965c5011001ff02b47ec45b568c92af8c60a6dd3

Páginas fuente

32

Diagramas Mermaid

6

Índice

[Inicio](#)

[Arquitectura](#)

[Evolución](#)

HOSTS

[Replicant](#)

[Nexus](#)

[DigitalOcean](#)

[GitHub](#)

RED

[Visión general](#)

[Inventario provisional](#)

DESPLIEGUE

[Modelos](#)

[Git](#)

[Docker](#)

OPERACIÓN

[SSH](#)

[Comandos útiles](#)

[Mantenimiento](#)

[Descargas](#)

Pendientes

APLICACIONES

[Índice](#)

[App Launch](#)

[Salones AV](#)

[Reserva-Pistas-UTP](#)

[Consumos Cupra](#)

[CV de Raúl](#)

[Control de Red](#)

[Cartera Estratégica](#)

[Replicant Lab](#)

[Autodocumentación](#)

[Decisiones](#)

CHANGE LOG

[Índice](#)

[13 de agosto de 2026](#)

[9 de agosto de 2026](#)

[8 de agosto de 2026](#)

Replicant Lab

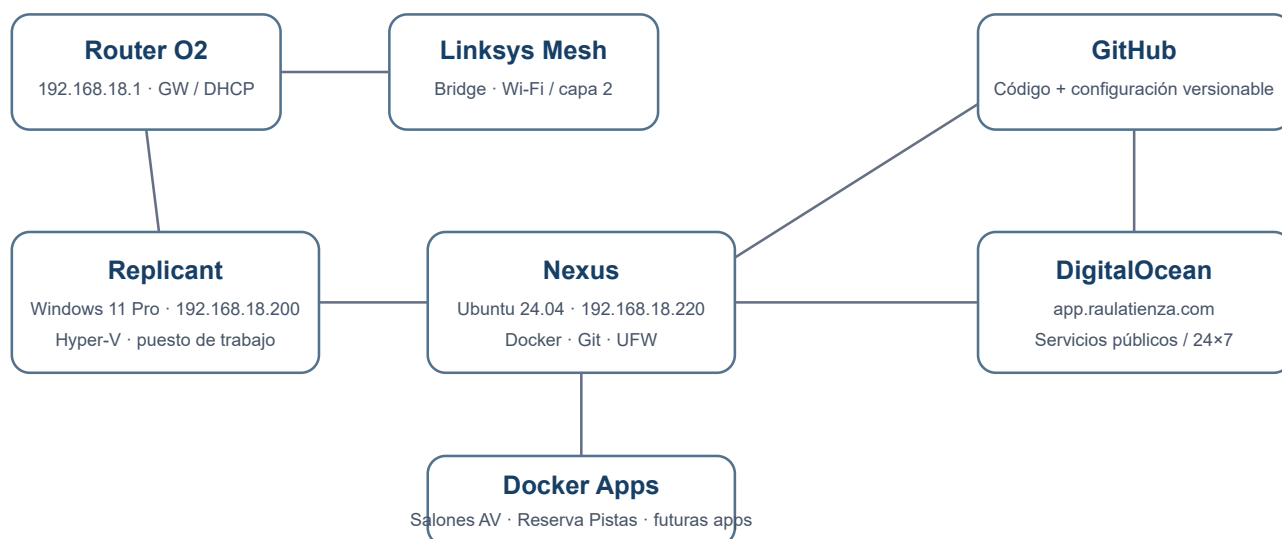
Manual vivo de la infraestructura local y cloud del laboratorio: qué existe, dónde se ejecuta, cómo se opera y qué queda pendiente.

Objetivo

Mantener una visión única, entendible y versionada de **concepto, hosts, red, seguridad, Git, Docker y operación**. La documentación funcional de cada aplicación vive en su propio repositorio.

Las versiones portables se descargan desde el icono **↓ Descargas** de la cabecera, junto al selector claro/oscuro.

Arquitectura de un vistazo



La portada usa **SVG nativo**, no Mermaid. Así evitamos que una incompatibilidad del parser pueda romper la vista principal.

Principios

- **Minimalismo:** pocas piezas y cada una con una función clara.
- **Git como fuente de verdad:** código y configuración versionable viven en GitHub.
- **Separación:** Windows para trabajo interactivo; Ubuntu para servicios; cloud para disponibilidad pública/24x7.
- **Persistencia fuera de Git:** datos, secretos y backups no se mezclan con repositorios.
- **Seguridad práctica:** SSH por clave, UFW y puertos publicados de forma explícita.
- **Reproducibilidad:** un host debe poder reconstruirse sin depender de cambios manuales no documentados.
- **Separación:** Checkout ≠ Runtime y Tarjeta App Launch ≠ Runtime local.

Estado actual

Componente	Estado
Replicant	✔ Operativo
Nexus	✔ Operativo
SSH por clave	✔ Operativo
UFW	✔ Activo
Docker / Compose	✔ Operativo
GitHub desde Nexus	✔ Operativo
App Launch	✔ Multientorno validado · Nexus 80 + DigitalOcean HTTP/HTTPS
Puerto 8080	✔ Libre y no asignado
Salones AV	✔ Operativa en Nexus · 8081 · Git/Nexus reconciliados en 8c0bc08
Replicant Lab en Nexus	✔ Runtime Nginx estático validado · 8082
Reserva-Pistas-UTP	✔ Validada y observada en Nexus · 8083
Cartera Estratégica	✔ MVP local en Replicant · no desplegada en Nexus
Reserva-Pistas histórico en Nexus	✔ 18 registros reconciliados bajo demanda · sin conflictos
Consumos Cupra	✔ Cloud Run · 9f66a368 · revisión 00009-pon al 100 %
CV	✔ Firebase Hosting · HTTP 200 · deuda POST-CARTERA
Control de Red	✔ Herramienta local · separación de datos POST-CARTERA
Producción DigitalOcean	✔ App Launch y Reservas; no es runtime de Consumos ni CV
Backups Nexus	⌚ POST-CARTERA
Nombres locales	✔ replicant y nexus mediante hosts en Replicant

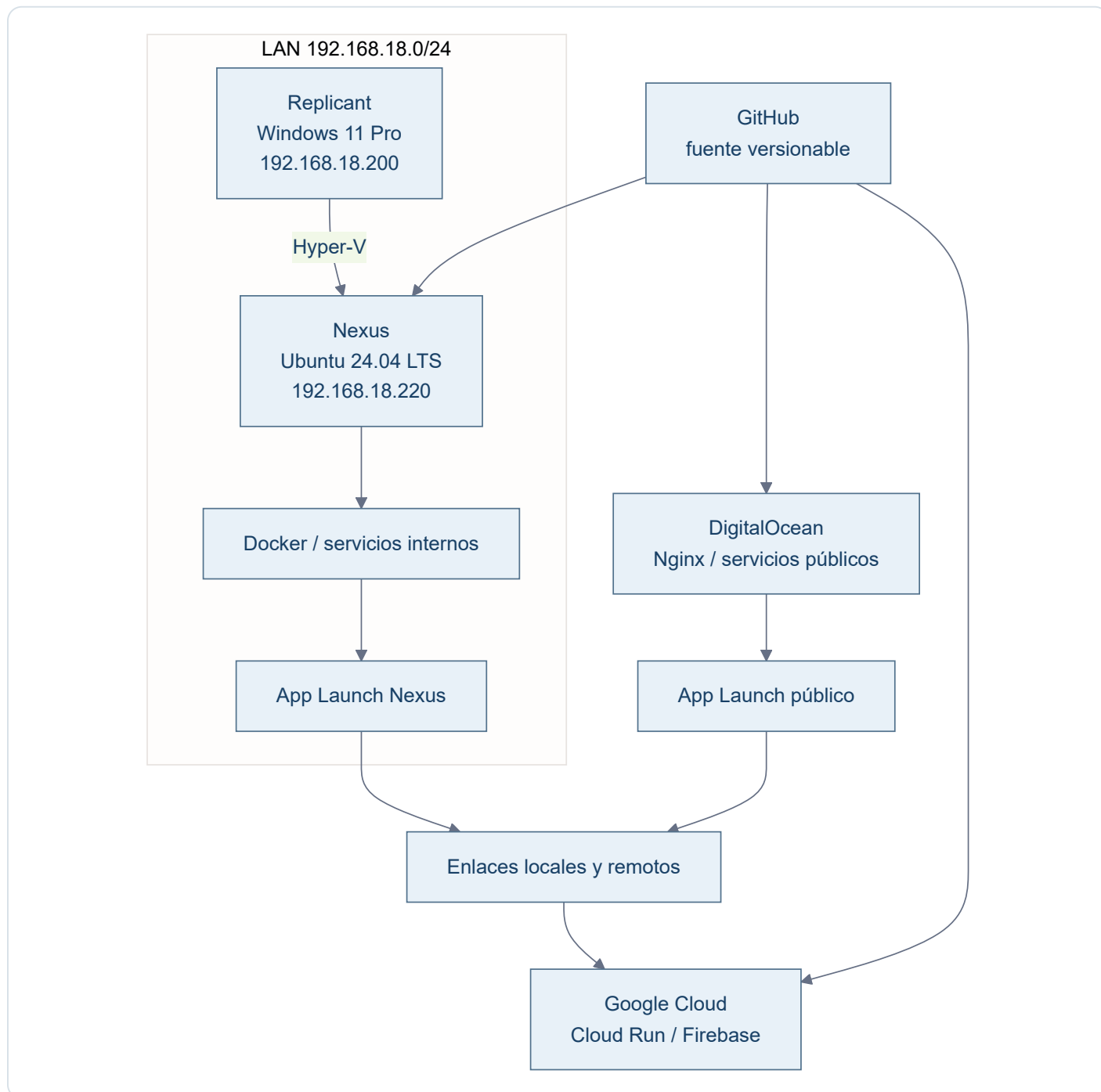
Cómo usar esta documentación

- **Arquitectura** explica cómo encajan las piezas.
- **Evolución** resume hitos y estado actual; el detalle histórico vive en Change Log.
- **Hosts** describe cada máquina.
- **Red** documenta direccionamiento e inventario.
- **Despliegue** explica los modelos heterogéneos: local, Docker, VPS, Cloud Run y Firebase.
- **Aplicaciones** contiene solo la ficha de infraestructura de cada app.
- **Operación** concentra comandos y procedimientos cortos.
- **Pendientes** conserva el estado y los encargos que deben retomarse sin depender de memoria de chat.
- **Decisiones** registra criterios que no conviene redescubrir cada vez.
- **Descargas** ofrece el documento global y las fichas HTML/PDF derivadas de cada aplicación.

Los estados distinguen entre contenido **implementado en Git**, **probado localmente**, **validado u observado en Nexus** y **validado en producción**. Una evidencia de un entorno no se extrapola a otro.

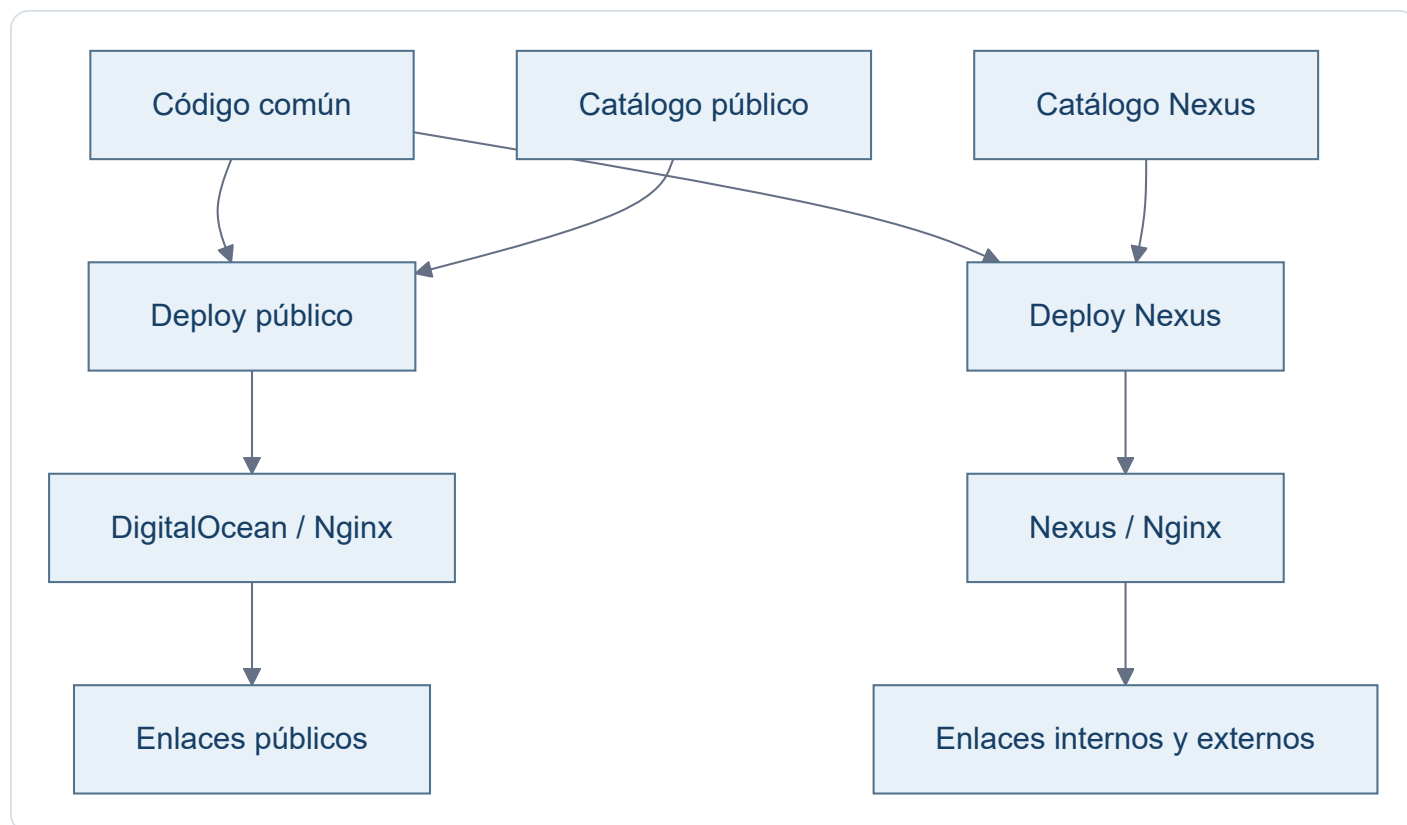
Arquitectura

Modelo general



App Launch es catálogo y capa de acceso. No ejecuta las aplicaciones enlazadas ni demuestra que residan en el mismo host.

App Launch multientorno



El catálogo se selecciona durante el despliegue. Cada host recibe únicamente su `apps.json`; la lógica visual es común.

Responsabilidades

Elemento	Responsabilidad
Replicant	Estación principal Windows, desarrollo local, Hyper-V y administración
Nexus	Laboratorio Linux, Docker, servicios internos y copias de consulta
GitHub	Código, configuración versionable e histórico
DigitalOcean	App Launch público, Reservas y otros servicios publicados en el VPS
Google Cloud Run	Producción canónica de Consumos Cupra
Firebase Hosting	Producción pública del CV
App Launch	Catálogo y navegación hacia aplicaciones locales o remotas
<code>/opt/data</code> , <code>/opt/secrets</code> , <code>/opt/backups</code>	Datos, secretos y copias fuera de Git

Dos reglas de lectura

Checkout ≠ Runtime y Tarjeta App Launch ≠ Runtime local.

Docker es el patrón preferido para servicios internos de Nexus cuando encaja, no un requisito universal para todas las aplicaciones.

Evolución del laboratorio

La historia detallada se conserva en el [Change Log](#). Para operar el laboratorio basta con estos hitos:

Etapa	Resultado
Construcción inicial	Replicant quedó como estación principal y host Hyper-V; Nexus como laboratorio Ubuntu/Docker.
Auditoría y normalización	Se inventariaron hosts, red, aplicaciones, checkouts y runtimes sin confundir presencia de código con ejecución.
Remediaciones	Salones AV quedó reconciliado en Git/Nexus; Consumos Cupra quedó trazado y restaurado en Cloud Run; el CV quedó identificado en Firebase Hosting.
Estado actual	Las fases 2A, 2B y 2C están cerradas. No hay una incidencia crítica abierta en esas aplicaciones.
Trabajo futuro	La deuda aceptada se concentra en Pendientes y se retomará después de Cartera Estratégica.

Criterio de validación

Cada aplicación se valida con los mecanismos apropiados para su tecnología: tests, lint, build, configuración, Compose, healthchecks, runtime o HTTP. La evidencia concreta vive en su [ficha técnica](#), sin duplicar aquí los listados de pruebas.

Host · Replicant

Campo	Valor
Host	Replicant
Rol	Estación de trabajo + host Hyper-V
SO	Windows 11 Pro
IP	192.168.18.200
Gateway	192.168.18.1
Virtualización	Hyper-V
Switch	Replicant Ethernet
Acceso	Sesión local de Windows; origen de la administración SSH

Responsabilidades

- Herramientas interactivas: ChatGPT, Codex, Gemini y similares.
- Administración de Nexus.
- Hyper-V.
- Punto de entrada SSH hacia Nexus y DigitalOcean.

Criterio

No convertir Replicant en servidor por comodidad si el servicio puede vivir limpiamente en Nexus.

Host · Nexus

Campo	Valor
Hostname	nexus
SO	Ubuntu Server 24.04.4 LTS
IP	192.168.18.220/24
Gateway	192.168.18.1
Interfaz	eth0
MAC	00:15:5d:12:7b:00
Plataforma	VM Hyper-V Gen2
Acceso	ssh raul@nexus
Función	Laboratorio Linux, Docker, servicios internos y copias de consulta

Software base

- OpenSSH Server.
- Docker Engine CE.
- Docker Compose.
- Git.
- UFW.
- Nano.
- unattended-upgrades.

Estructura

/opt/apps

repositorios y aplicaciones

/opt/data

datos persistentes

/opt/backups

copias

/opt/compose

composiciones separadas si hacen falta

/opt/secrets

secretos y .env fuera de Git

Seguridad

Estado

SSH por clave, UFW activo y actualizaciones automáticas de seguridad habilitadas.

Puertos conocidos

Puerto	Uso	Ámbito
22/tcp	SSH	LAN
80/tcp	App Launch	LAN
8080/tcp	Libre	Sin listener
8081/tcp	Salones AV	LAN
8082/tcp	Replicant Lab · Nginx estático	LAN
8083/tcp	Reserva-Pistas-UTP · Nginx	LAN
53	systemd-resolved	localhost

Estado observado el 13/08/2026

- Docker y `unattended-upgrades` activos.
- Replicant Lab ejecutándose como sitio estático en Nginx, construido desde el `Dockerfile` y publicado en `192.168.18.220:8082` → `80`, sin bind mounts ni servidor de desarrollo.
- Salones AV accesible mediante Nginx en `192.168.18.220:8081`; checkout limpio e idéntico a GitHub `main` en `8c0bc08` después de la Fase 2A.
- Reserva-Pistas-UTP ejecutándose como backend privado y proxy Nginx en `192.168.18.220:8083`, con autenticación, datos persistentes separados y canal saliente SSH restringido hacia DigitalOcean.
- App Launch ejecutándose en el puerto `80` mediante Nginx `1.27-alpine`, con sitio y configuración montados en solo lectura.
- `8080` sin listener.
- CV y Control de Red presentes solo como checkouts; no son servicios Nexus.
- Consumos Cupra y Cartera Estratégica sin checkout servido ni puerto Nexus.
- Una tarjeta de App Launch puede apuntar a una aplicación externa; no implica ejecución en Nexus.

Esta observación confirma el estado del laboratorio privado en esa fecha. DigitalOcean se validó por separado y de forma acotada para el cierre de Reserva-Pistas-UTP; cada repositorio conserva sus definiciones operativas.

Runtime documental validado en Nexus

El PR #9 fusionado sustituye `mkdocs serve` y los bind mounts por una imagen construida desde el `Dockerfile`: MkDocs estricto en la etapa builder y Nginx estático en runtime, manteniendo `192.168.18.220:8082`. El 09/08/2026 se reconstruyó y recreó exclusivamente este servicio en Nexus y se validaron HTTP, navegación, recursos, cinco diagramas Mermaid y descargas HTML/PDF idénticas byte a byte a los artefactos versionados. El HTML funciona offline sin dependencias esenciales externas y el PDF conserva la documentación completa.

Host · DigitalOcean

Campo	Valor
Alias SSH	docean
Host	app.raulatienza.com
SO	Linux en VPS DigitalOcean
Acceso	SSH administrativo por clave
Función	Nginx, App Launch público y servicios web alojados en el VPS

Modelo operativo

DigitalOcean publica App Launch como capa de navegación y aloja Reserva-Pistas-UTP. Debe mantener despliegues desde fuentes versionadas, secretos fuera de Git y mínima configuración manual.

App Launch puede enlazar servicios externos. En particular, Consumos Cupra se ejecuta en Google Cloud Run y el CV en Firebase Hosting; sus tarjetas no los convierten en runtimes de DigitalOcean.

Estado validado

- App Launch respondió 200 por HTTP/HTTPS y sus assets y catálogo público fueron verificados.
- Reserva-Pistas-UTP mantiene backend local y publicación mediante Nginx con autenticación.
- El canal de sincronización de Reservas con Nexus usa una clave dedicada y un comando SSH forzado, sin shell, TTY ni forwarding.
- No existe sincronización automática de los datos de Reservas: el operador revisa una vista previa y confirma una dirección.

La documentación no incluye contraseñas, claves privadas ni valores de secretos. Los cambios de DNS, certificados, servicios o datos requieren un encargo específico.

Host lógico · GitHub

GitHub no es un host de ejecución del lab, pero sí una pieza de infraestructura crítica porque actúa como **fuentes de verdad**.

Reglas

- Código y configuración versionable viven aquí.
- Cambios relevantes se realizan en rama.
- Se revisan mediante Pull Request.
- `main` representa el estado estable aprobado.
- Datos, bases de datos, `.env`, tokens y claves privadas no se versionan.

Repos relevantes

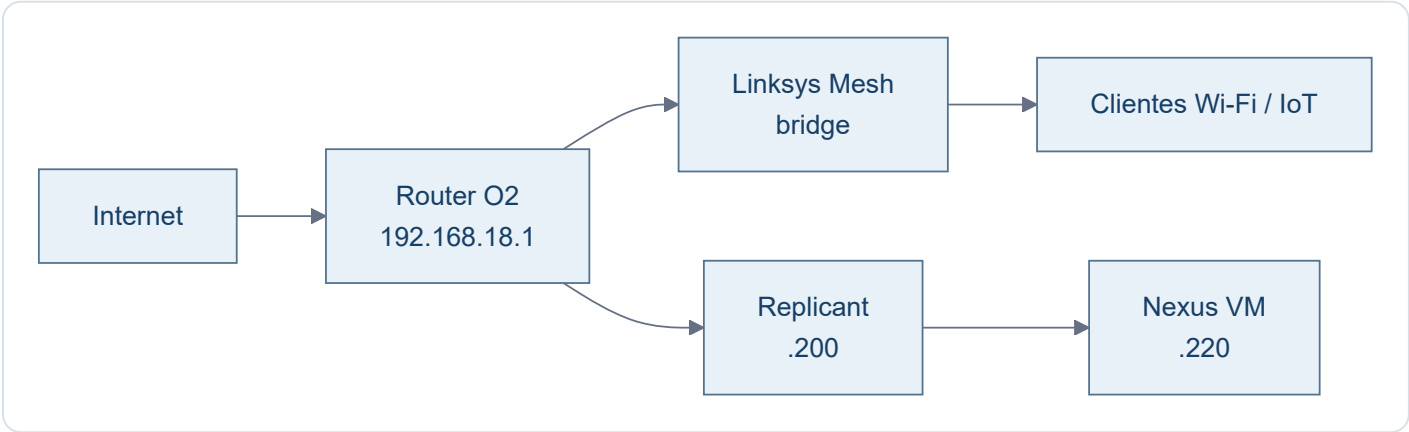
Repo	Función
ratienza/infra-replicant-lab	Documentación viva de infraestructura
ratienza/salones-av-valencia-palace	Aplicación AV / PoC Docker
ratienza/cartera-estrategica	Aplicación de cartera
ratienza/Reserva-Pistas-UTP	Aplicación de reservas de pádel
ratienza/Apps_Lauch	Catálogo público e interno de aplicaciones
ratienza/Consumos_Cupra	Aplicación de consumos desplegada en Cloud Run
ratienza/CV-Raul-IA-Estudio-Google-	Fuente del CV publicado en Firebase Hosting
ratienza/control-red	Herramienta local de inventario de red

Cada aplicación conserva en su propio repositorio su código, configuración reproducible y documentación funcional. Este repositorio solo mantiene la visión de infraestructura y las diferencias comprobadas entre entornos.

Un checkout no prueba que exista un runtime, y un trigger conectado a GitHub no se considera producción si la cadena real no está demostrada.

Red · Visión general

Topología



Convención provisional

Rango	Uso previsto
.1	Router / gateway
.2 - .19	Infraestructura de red / mesh
.20 - .179	Clientes, IoT y DHCP general
.180 - .199	Equipos fijos, NAS, impresoras
.200 - .219	Servidores físicos
.220 - .239	VMs / laboratorio
.240 - .254	Reserva futura

DNS

No existe servidor DNS local. Replicant mantiene dos aliases puntuales en el archivo `hosts` de Windows:

Nombre	IP	Uso
replicant	192.168.18.200	Escritorio remoto y herramientas del host
nexus	192.168.18.220	SSH y <code>http://nexus/</code>

El alias solo resuelve el nombre. El protocolo, puerto, usuario y credenciales siguen perteneciendo a cada servicio.

Red · Inventario provisional

Estado

Inventario de trabajo obtenido del escaneo de agosto de 2026. No se considera una CMDB definitiva y algunos nombres/fabricantes pueden requerir validación posterior.

Infraestructura y equipos conocidos

IP	Nombre / función	MAC / fabricante / nota
.1	Router Fibra O2	2C-96-82-69-25-F2 · gateway
.3	RAN-CASA	HP
.4	Sonos 5	5C-AA-FD-0D-54-36 · no visto
.5	Enchufe Emma	4C-11-AE-14-10-6E · IoT
.6	Keres.local	14-91-82-8D-8C-6A · Linksys
.7	SONOS Cocina	B8-E9-37-E7-51-2E
.8	Enchufe ordenador RAN-CASA	D4-A6-51-E5-0F-24 · Tuya
.9	Termostato Nest	18-B4-30-C4-52-48
.10	Sonos Habitación	B8-E9-37-E1-8E-AE
.11	Echo Darío	F4-03-2A-7B-1B-1B
.12	Alexa Comedor	DC-91-BF-D1-35-B3
.13	Linksys13497.local	14-91-82-8D-8C-B2 · Linksys
.14	Alexa Despacho	20-A1-71-00-43-44
.18	Play Darío	0C-FE-45-37-AA-4F · Sony
.19	Alexa Cocina	1C-FE-2B-4F-5A-1B
.20	Fire Stick	34-AF-B3-DE-58-06
.25	IoT desconocido	28-6D-CD-49-AF-28 · no visto
.27	Lamparita Darío	28-6D-CD-56-C8-41
.35	Lamparita comedor	28-6D-CD-5D-BD-81
.42	Chromecast antiguo	8A-05-36-D5-F8-2D · no visto
.44	Escalera	40-F5-20-C1-C7-FB · luces · no visto
.45	Lamparita comedor escalera	28-6D-CD-49-AB-D1

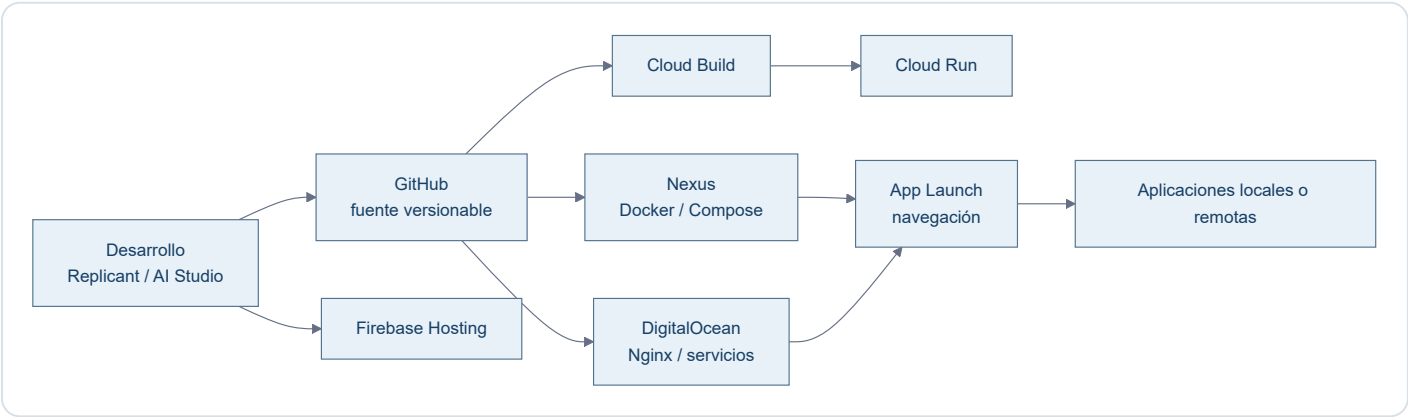
IP	Nombre / función	MAC / fabricante / nota
.52	Enchufe Raúl	D4-A6-51-E6-94-8B
.53	Sebastian - CECOTEC	44-87-63-97-02-EA
.54	Impresora Darío	28-C5-C8-AB-D7-BE · HP
.71	Luz despacho	CE-CC-5A-B6-E9-BC · no visto
.74	Móvil Emma	76-9D-67-FE-0D-CA · Samsung · no visto
.82	Chromecast	F6-45-CE-C2-7C-F6
.83	Móvil Darío	BA-D8-D1-F2-04-44 · no visto
.94	Portátil SH	28-92-00-45-01-CD · HP · no visto
.101	Móvil Emma antiguo	58-02-05-B8-4E-BA · no visto
.106	Móvil Raúl	52-1E-45-1D-FB-AA
.123	Antigua IP DHCP de Replicant	00-E0-4C-4B-35-AD · migrado a .200
.124	No identificado	A4-E8-8D-08-12-44 · no visto
.125	Antigua IP DHCP de Nexus	00-15-5D-12-7B-00 · migrado a .220
.200	Replicant	host físico Windows / Hyper-V · IP fija
.220	Nexus	VM Ubuntu / Docker · IP fija

Nota operativa

El inventario sirve para orientación y futuras decisiones de direccionamiento. No se reubicarán dispositivos por estética ni se convertirán en IP fija salvo necesidad concreta.

Modelos de despliegue

Replicant Lab es deliberadamente heterogéneo: el runtime se elige por las necesidades de cada aplicación y no por una regla única.



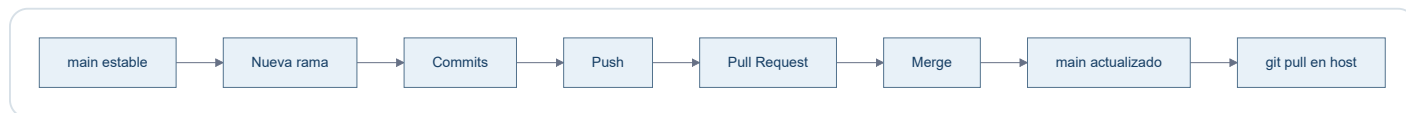
Aplicación	Creación / desarrollo	Deploy	Runtime
Salones AV	Codex / GitHub	Docker Compose	Nexus
Reserva-Pistas-UTP	Codex / GitHub	Compose en Nexus y despliegue controlado en VPS	Nexus + DigitalOcean
Consumos Cupra	AI Studio + Codex / GitHub	Cloud Build + Artifact Registry	Google Cloud Run
CV	AI Studio / GitHub	Despliegue manual observado	Firebase Hosting
Cartera Estratégica	Codex / desarrollo local	Ejecución local	Replicant
Control de Red	PowerShell + Codex	Ejecución local	Replicant
App Launch	Codex / GitHub	Scripts por destino	Nginx en Nexus y DigitalOcean
Replicant Lab	Codex / GitHub	Docker Compose	Nexus

Principios

- Checkout ≠ Runtime : un repositorio presente en un host puede ser solo una copia de consulta.
- Tarjeta App Launch ≠ Runtime local : una tarjeta es un enlace y no demuestra dónde se ejecuta la aplicación.
- App Launch es una capa de navegación; la autenticación, los datos y el ciclo de vida pertenecen a cada aplicación.
- Docker es el patrón principal en Nexus, no el único modelo del laboratorio.

Despliegue · Git

Flujo normal



Convención de ramas para esta documentación

Las ramas representan **cambios lógicos**, no archivos individuales.

Ejemplos:

- docs/bootstrap-infra
- docs/app-salones
- docs/app-cartera
- docs/network
- docs/backups

Regla

Main

`main` debe representar siempre la documentación aprobada y coherente con el estado real.

Despliegue · Docker

Este patrón describe los servicios Docker de Nexus. No se extrapola a Cloud Run, Firebase Hosting, servicios del VPS ni aplicaciones locales; la comparación completa está en [Modelos de despliegue](#).

Patrón

```
GitHub
↓
/opt/apps/<repo>
↓
compose.yml
↓
docker compose up -d
↓
servicios
```

Buenas prácticas del lab

- Preferir imágenes oficiales cuando sea razonable.
- Minimizar paquetes instalados en el host.
- Persistencia fuera del contenedor.
- Secretos fuera de Git.
- Publicar solo puertos necesarios.
- Cuando proceda, ligar puertos a `192.168.18.220` en vez de `0.0.0.0`.
- Separar proxy, backend y datos cuando cada pieza tenga una responsabilidad clara.
- Ejecutar los procesos de aplicación con un usuario no root siempre que sea viable.
- Usar `restart: unless-stopped` para servicios que deben recuperarse tras reiniciar Nexus.
- No añadir paneles de administración si no resuelven un problema real.

Convención de puertos en Nexus

Puerto	Servicio	Estado
80	App Launch Nexus	Operativo
8080	Sin asignar	Libre
8081	Salones AV	Operativo
8082	Replicant Lab · documentación	Operativo
8083	Reserva-Pistas-UTP	Operativo
8084+	Próximos servicios	Asignación secuencial

Reglas:

- App Launch usa el puerto estándar `80`, sin puerto explícito en la URL.
- `8080` queda libre y no se reserva para App Launch.
- A partir de `8081`, los servicios se numeran consecutivamente salvo necesidad técnica justificada.
- Cada nuevo servicio debe quedar documentado aquí al asignar su puerto.

- Siempre que sea posible, publicar el servicio ligado a `192.168.18.220` y no a `0.0.0.0`.

App Launch es la excepción deliberada: publica `80` en todas las interfaces de Nexus para actuar como entrada de la LAN. No se publica por HTTPS.

Salones AV aplica explícitamente la regla LAN mediante `192.168.18.220:8081:80`. El PR `salones-av-valencia-palace#2` incorporó el bind a `main`; desde `8c0bc08` el checkout Nexus está limpio y la configuración es reproducible desde GitHub.

Reserva-Pistas-UTP · patrón validado

Reserva-Pistas utiliza un Compose de dos servicios:

```
LAN :8083
  ↓
nginx
  ↓
red privada Compose
  ↓
app:8765
```

El backend no publica `8765` hacia el host. Nginx lo alcanza mediante el DNS interno de Docker usando el nombre de servicio `app`.

Los datos se desacoplan del ciclo de vida del contenedor mediante:

```
/opt/data/reserva-pistas:/app/data
```

La aplicación recibe por entorno:

```
APP_HOST=0.0.0.0
APP_PORT=8765
APP_DATA_DIR=/app/data
```

El código conserva valores por defecto compatibles con despliegues no Docker.

Operación estándar

```
cd /opt/apps/<proyecto>
git switch main
git pull
docker compose up -d --build
```

Comprobaciones habituales:

```
docker compose ps
docker compose logs --tail 50
```

Parada y recreación:

```
docker compose down
docker compose up -d
```

Un `down` elimina contenedores y la red del proyecto, pero no debe eliminar datos persistentes alojados fuera del contenedor.

Autoarranque

Docker Engine arranca con Ubuntu. Los contenedores existentes con `restart: unless-stopped` se recuperan automáticamente después de reiniciar Nexus.

Compose no necesita ejecutarse manualmente durante el arranque para recuperar esos contenedores ya creados; su fichero YAML sigue siendo la definición reproducible para recrearlos o actualizarlos.

Replicant Lab · sitio estático reproducible

La definición versionada converge en un único modelo:

```
mkdocs.yml + docs/
↓
Dockerfile · MkDocs build --strict
↓
sitio estático
↓
Nginx :80
↓
192.168.18.220:8082
```

`compose.yml` construye el `Dockerfile`, publica únicamente `192.168.18.220:8082 → 80` y no monta fuentes ni ejecuta `mkdocs serve`.

Actualización desplegada

```
cd /opt/apps/infra-replicant-lab
git switch main
git pull --ff-only origin main
docker compose up -d --build
```

Cada cambio documental desplegado requiere reconstruir la imagen porque Nginx sirve la copia estática incluida durante el build. Las dependencias viven en las etapas versionadas; no se instalan en Nexus.

Separación de estados

El modelo estático está implementado, probado localmente y validado en Nexus. La validación del 09/08/2026 incluyó el contenedor Nginx estable, HTTP y navegación, cinco diagramas Mermaid, descargas reales idénticas a Git, HTML offline y PDF completo. No implica validación en producción.

Operación · SSH

Desde Replicant

```
ssh raul@nexus  
ssh docean
```

`ssh raul@nexus` es la referencia operativa principal. El alias corto puede depender de la configuración SSH local y no debe darse por supuesto.

Configuración conceptual

```
Host nexus  
  HostName 192.168.18.220  
  User raul  
  IdentityFile C:\Users\raul\.ssh\nexus_local  
  IdentitiesOnly yes  
  
Host docean  
  HostName app.raulatienda.com  
  User root  
  IdentityFile C:\Users\raul\.ssh\reserva_pistas_do  
  IdentitiesOnly yes
```

Claves

- `nexus_local` : Replicant → Nexus.
- `reserva_pistas_do` : Replicant → DigitalOcean.
- `~/.ssh/id_ed25519` en Nexus: Nexus → GitHub.

Nunca documentar

No almacenar en este repo claves privadas, tokens, contraseñas ni el contenido de secretos.

Operación · Comandos útiles

Host

```
ip addr  
ip route  
sudo ss -tulpn
```

Firewall

```
sudo ufw status verbose
```

Docker

```
docker ps  
docker compose up -d  
docker compose down  
docker compose logs
```

Git

```
git status  
git switch main  
git pull
```

Actualizaciones

```
sudo apt update  
apt list --upgradable  
sudo apt upgrade
```

Filosofía

Este documento evita convertirse en un recetario infinito. Solo se incluyen comandos frecuentes y útiles para operar el lab.

Operación · Mantenimiento

Actualizaciones

`unattended-upgrades` está activo y habilitado. Ubuntu puede diferir ciertos paquetes mediante phased rollout; no se fuerzan salvo necesidad.

Comprobación:

```
systemctl status unattended-upgrades --no-pager
```

Limpieza

`apt autoremove` puede utilizarse tras revisar qué paquetes se eliminarán.

Criterio de mantenimiento

- Actualizar con regularidad.
- No instalar herramientas "por si acaso".
- Revisar servicios expuestos con `ss -tulpn`.
- Mantener `main` coherente con la realidad.

Descargas

La documentación MkDocs de este proyecto es la única referencia canónica. Los dos ficheros siguientes se generan exclusivamente desde `mkdocs.yml`, las páginas de `docs/` incluidas en `nav` y sus recursos versionados.

↓ Descargar documentación HTML offline

↓ Descargar documentación PDF

Rutas canónicas únicas

Artefacto	Ruta	Cobertura
HTML autocontenido	<code>docs/downloads/Replicant-Lab.html</code>	Toda la navegación MkDocs, índice interno y seis Mermaid
PDF A4	<code>docs/downloads/Replicant-Lab.pdf</code>	Portada, índice, documentación completa, numeración y seis Mermaid

No se mantiene ninguna copia alternativa.

Fichas técnicas individuales

Cada pareja HTML/PDF se genera desde la misma página Markdown incluida en la navegación. El HTML funciona offline y el PDF está preparado para consulta o archivo.

Aplicación	HTML	PDF
App Launch	Descargar	Descargar
Salones AV	Descargar	Descargar
Reserva-Pistas-UTP	Descargar	Descargar
Consumos Cupra	Descargar	Descargar
CV de Raúl	Descargar	Descargar
Control de Red	Descargar	Descargar
Cartera Estratégica	Descargar	Descargar
Replicant Lab	Descargar	Descargar

Propiedades verificables

- Todos los artefactos muestran la misma huella SHA-256 de las fuentes.
- El HTML incorpora CSS, JavaScript y Mermaid localmente y funciona mediante `file://` sin red.
- El HTML no solicita CDN ni contiene clientes de recarga, conexiones dinámicas o referencias al servidor de desarrollo.
- El PDF mantiene texto seleccionable, enlaces, tablas, código, avisos y diagramas.
- El JavaScript del sitio calcula las rutas desde su propio recurso y funciona con el sitio estático en `/`.

Regeneración y sincronía

Después de preparar las dependencias fijadas, un único comando regenera y valida todo:

```
python scripts/docs_pipeline.py generate
```

CI ejecuta el modo `check`, vuelve a renderizar un PDF de control, verifica la huella y cobertura, construye la imagen Docker final, arranca Nginx temporalmente y compara byte a byte las descargas servidas con los artefactos versionados.

Estado de despliegue

El pipeline y el runtime estático están implementados y probados localmente. El 13/08/2026 se comprobaron las 22 rutas del sitio, seis diagramas, el dossier global y las ocho parejas de fichas individuales; los ficheros servidos por el entorno de validación coincidieron byte a byte con los artefactos generados. La publicación en Nexus se valida de nuevo después de integrar el cambio en `main`; esta evidencia corresponde al laboratorio privado y no se extrapola a producción.

Pendientes

No hay incidencias críticas abiertas en Salones AV, Consumos Cupra o CV. Las fases 2A, 2B y 2C están cerradas.

Activos

- Continuar Cartera Estratégica como siguiente bloque de producto.

Deuda POST-CARTERA

CV / Firebase

- Reconciliar Firebase y Cloud Build.
- Retirar o corregir el trigger y la configuración obsoletos.
- Demostrar trazabilidad entre SHA Git y versión Firebase.
- Incorporar lockfile y build reproducible, CI, tests y typecheck.
- Revisar restos React, metadata Gemini y artefactos residuales.
- Revisar cabeceras de seguridad, caché, protección de `main` y UX de revelado de contacto.

Control de Red

- Hacer backup antes de intervenir.
- Separar inventarios, snapshots y datos operativos reales del código Git.
- Mantener únicamente ejemplos anonimizados en el repositorio.

Nexus

- Definir una política global de backups y ejecutar una prueba de restauración.
- Inventariar y gestionar los checkouts que son solo copias de consulta.
- Mantener explícito `Checkout ≠ Runtime`; el CV en Nexus no es un despliegue.

Mejoras opcionales

- Limpiar assets PWA históricos no utilizados de App Launch mediante un cambio versionado y verificable.

La deuda aceptada no debe presentarse como fallo de producción ni reabrir las fases ya cerradas.

Aplicaciones

Vista operativa compacta. Cada nombre abre su ficha técnica completa.

Aplicación	Objetivo	Stack / herramienta	Repositorio	Deploy / runtime	Estado
App Launch	Catálogo de acceso	HTML/CSS/JS · Codex	<code>ratienza/Apps_Launch</code>	Scripts · Nginx en Nexus y DigitalOcean	Operativo
Salones AV	Guía audiovisual	HTML · Codex	<code>ratienza/salones-av-valencia-palace</code>	Compose · Nexus 8081	Cerrado / operativo
Reserva-Pistas-UTP	Reservas de pádel	Python/Flask · Codex	<code>ratienza/Reserva-Pistas-UTP</code>	Compose + systemd/Nginx · Nexus y DigitalOcean	Operativo
Consumos Cupra	Registro y análisis de consumos	React/Vite/Express · AI Studio + Codex	<code>ratienza/Consumos_Cupra</code>	Cloud Build · Cloud Run	Cerrado / operativo
CV de Raúl	CV y portfolio público	Vite/Tailwind · AI Studio	<code>ratienza/CV-Raul-IA-Estudio-Google-</code>	Firebase Hosting	Operativo / POST-CARTERA
Control de Red	Inventario de LAN	PowerShell · Codex	<code>ratienza/control-red</code>	Local · Replicant	Operativo local / POST-CARTERA
Cartera Estratégica	Gestión de inversión	Python/Streamlit · Codex	<code>ratienza/cartera-estrategica</code>	Local · Replicant	MVP operativo
Replicant Lab	Manual vivo del laboratorio	MkDocs/Mermaid · Codex	<code>ratienza/infra-replicant-lab</code>	Compose · Nexus 8082	Operativo

Lectura correcta

- Un checkout en Nexus puede ser solo una copia de consulta.
- App Launch enlaza aplicaciones; una tarjeta no acredita runtime local.
- Los detalles de build, persistencia, seguridad, pruebas, rollback y deuda viven en cada ficha.

Aplicación · App Launch

Catálogo y capa de navegación para acceder a aplicaciones públicas y servicios internos. App Launch no es el runtime de las tarjetas: la presentación es única y cada entorno recibe exclusivamente su catálogo activo.

Estado auditado

Campo	DigitalOcean	Nexus
Desarrollo	Codex · HTML/CSS/JavaScript	Mismo origen
Repositorio	ratienza/Apps_Lauch · main · ced6e149315ac14338b547a6773dbb817270b84a	Mismo origen
URL	http://app.raulatienza.com y https://app.raulatienza.com	http://192.168.18.220
Publicación	Nginx del host · /opt/portal	Compose app-launch · Nginx 1.27-alpine
Puerto	80/443 , sin puerto explícito	80 → 80/tcp
Catálogo activo	catalogs/public.json → apps.json	catalogs/nexus.json → apps.json
Validación	HTTP/HTTPS, HTML, PNG y JSON: 200	HTML, PNG y JSON: 200 ; servicios enlazados sanos

Validación final: **13/08/2026**. El repositorio, el checkout local y GitHub coincidían en el SHA indicado.

Arquitectura y funcionamiento

index.html carga ./apps.json en tiempo de ejecución y crea las tarjetas mediante nodos DOM. Nombre, descripción y acción se asignan con textContent ; el manifiesto no se interpreta como HTML.

El despliegue publica solo index.html , el fondo y el catálogo seleccionado renombrado como apps.json . DigitalOcean no recibió catalogs/nexus.json , direcciones 192.168.18.220 , puertos 8081-8083 ni nombres exclusivos del laboratorio.

En Nexus el contenedor monta en solo lectura:

```
/home/raul/app-launch/site      → /usr/share/nginx/html
/home/raul/app-launch/nginx.conf → /etc/nginx/conf.d/default.conf
```

Usa una red Compose propia, no tiene persistencia funcional y se recupera con restart: unless-stopped .

Catálogos comprobados

El catálogo público contiene Reservas, Consumos Cupra, Multimedia VPalace y CV. El catálogo Nexus contiene Replicant Lab, Reservas, Multimedia VPalace y enlaces externos a Consumos y CV. Un enlace presente en el catálogo interno no implica que su aplicación se ejecute en Nexus.

Regla

Tarjeta App Launch ≠ Runtime local.

Actualización y rollback

```
powershell -ExecutionPolicy Bypass -File .\deploy\deploy.ps1 -Target public
powershell -ExecutionPolicy Bypass -File .\deploy\verify.ps1 -Target public
powershell -ExecutionPolicy Bypass -File .\deploy\deploy.ps1 -Target nexus
powershell -ExecutionPolicy Bypass -File .\deploy\verify.ps1 -Target nexus
```

La actualización permanente debe partir de `main`. Para rollback se revierte el cambio en Git mediante una rama/PR y se vuelve a desplegar el destino afectado; no se copian ficheros desde un host hacia GitHub.

Seguridad y límites

- El repositorio no contiene claves ni configuración local real; `deploy/local.settings.ps1` está ignorado.
- La portada pública no solicita contraseña. La autenticación pertenece a cada aplicación enlazada.
- Nexus es HTTP de LAN; no se declara HTTPS interno.
- El puerto `8080` está libre y no pertenece a App Launch.
- El despliegue público conserva algunos assets PWA históricos no versionados en `/opt/portal/assets`; no intervienen en la portada actual y quedan como deriva a limpiar solo mediante un cambio Git deliberado.

Pruebas realizadas

- Verificación versionada `public`: catálogo exacto, ausencia de referencias Nexus y HTTP/HTTPS correctos.
- Verificación versionada `nexus`: catálogo exacto y salud HTTP de `8081`, `8082` y `8083`.
- `nginx -t` correcto en DigitalOcean.
- Comprobación visual automatizada de escritorio y móvil incluida en el cierre documental.

Aplicación · Salones AV

Documentación operativa audiovisual para el personal del SH Valencia Palace: panel general, pantallas LED, traslado, conexión de cliente, sonido y mapas de las plantas PL1 y PL6.

Ficha de infraestructura

Campo	Valor
Desarrollo	Codex / GitHub
Repo	ratienza/salones-av-valencia-palace
Host	Nexus
Ruta	/opt/apps/salones-av-valencia-palace
Contenedor	salones-av
Imagen	nginx:alpine
Puerto	192.168.18.220:8081 → 80/tcp
Contenido	proyecto_html montado en solo lectura
Rama / SHA	main · 8c0bc08446256974b7efa57c730cbeb1b6e81520
Red	Compose propia, sin dependencias con otros contenedores
Inicio	restart: unless-stopped

Comportamiento de despliegue

El contenido HTML se sirve mediante un bind mount. Por tanto, un `git pull` actualiza los ficheros visibles sin necesidad de reconstruir una imagen.

La actualización segura exige revisar primero el working tree, actualizar desde `main` y comprobar enlaces y páginas. El rollback consiste en volver a un commit aprobado y restaurar el contenido estático; no tiene base de datos ni persistencia funcional.

Reconciliación Git y Nexus

La Fase 2A corrigió mediante el PR `salones-av-valencia-palace#2` la única deriva del checkout: `main` ahora versiona `192.168.18.220:8081:80`. El cambio se reconstruyó conscientemente en una rama; no se copió el checkout del servidor hacia GitHub.

Antes del avance rápido se demostró que el blob preparado, el archivo vivo y el stash preservativo eran idénticos: `686cfdd506fbd67b624bffa5f49ca640a799da6`. Después del merge, GitHub `main`, `origin/main` y el checkout Nexus quedaron en `8c0bc08446256974b7efa57c730cbeb1b6e81520`, sin cambios locales.

Validación del 13/08/2026

- Contenedor `salones-av` activo con imagen `nginx:alpine` .
- `http://192.168.18.220:8081/` respondió `200` .
- Las siete páginas del menú respondieron `200` .
- Los enlaces HTML locales pasaron la comprobación estática.
- `docker compose config --quiet` aceptó la definición versionada.
- Docker confirmó el bind efectivo `192.168.18.220:8081` ; no existe publicación de Salones en `0.0.0.0:8081` .
- GitHub `main` == `checkout Nexus` en `8c0bc08` tras el despliegue dirigido al servicio `web` .
- La copia pública `https://app.raulatienza.com/salones/` respondió `200` .
- No se validaron equipos AV físicos, números de tomas ni procedimientos sobre hardware.

Seguridad y límites

El contenido operativo puede incluir topología e inventario de salas. No deben incorporarse credenciales ni datos de red sensibles a un repositorio público. El servicio es estático y no implementa autenticación propia; su alcance depende del bind LAN y del control del host.

Alcance de esta ficha

La documentación técnica/funcional específica de los salones permanece en el repo de la aplicación.

Reserva-Pistas-UTP

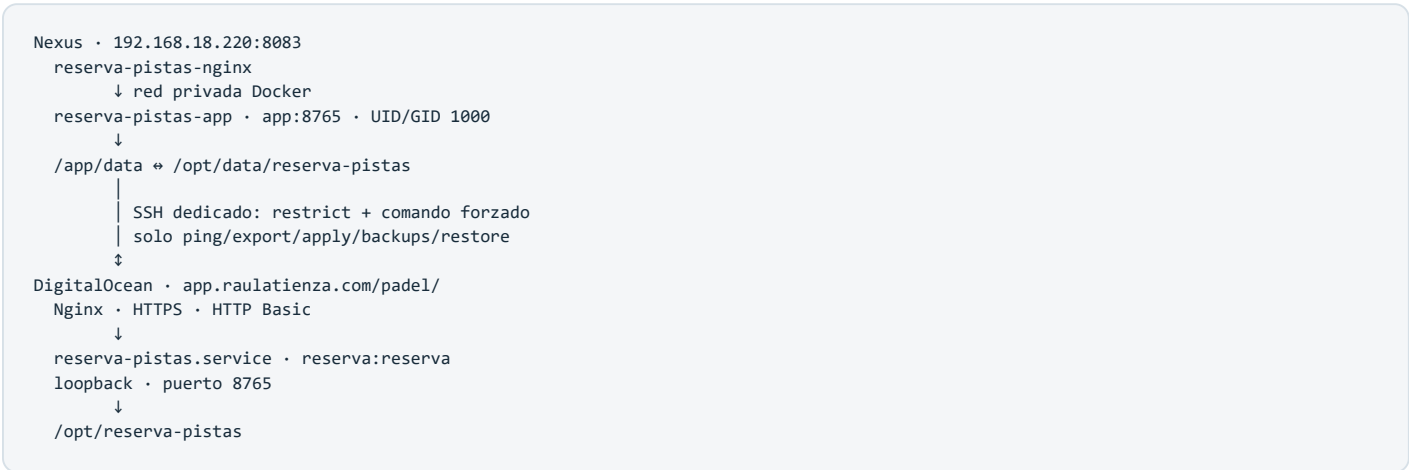
Ficha de infraestructura y operación de la aplicación de reservas de pádel de Torre de Porta-Coeli. La documentación funcional y de desarrollo permanece en `ratienza/Reserva-Pistas-UTP` ; esta página registra el montaje real en Raul Lab y producción.

Estado validado

Elemento	Valor
Desarrollo	Codex / GitHub
Repositorio	<code>ratienza/Reserva-Pistas-UTP</code>
<code>main</code> validado	<code>6fb055e16eb232d47cc2d759bed0ce2caff4cec9</code> (runtime) + <code>6df1698d3346db5c35f7d173b5d7dc2567ce8a5e</code> (guía operativa)
Nexus	✅ Docker Compose operativo
Ruta / URL Nexus	<code>/opt/apps/Reserva-Pistas-UTP</code> · <code>http://192.168.18.220:8083</code>
Datos Nexus	<code>/opt/data/reserva-pistas</code> → <code>/app/data</code>
DigitalOcean	✅ <code>reserva-pistas.service</code> en <code>loopback:8765</code>
Ruta / URL producción	<code>/opt/reserva-pistas</code> · <code>https://app.raulatienza.com/padel/</code>
Publicación	Nginx + HTTPS + HTTP Basic existente
Sincronización	✅ Bidireccional, por registros y exclusivamente bajo demanda
Issue de handover	#2 cerrado con evidencias

Validación de sincronización: **10/08/2026**. Revalidación de servicio y código: **13/08/2026**. No se cambiaron puertos, UFW, DNS, certificados, Nginx, autenticación pública ni otros servicios.

Arquitectura real



Nexus coordina ambas direcciones. La clave dedicada está montada de solo lectura, la huella del host DigitalOcean fue contrastada con el canal SSH administrativo ya confiable y el `authorized_keys` remoto no concede shell, TTY ni forwarding. El comando forzado baja privilegios a `reserva` antes de ejecutar `sync_peer.py` .

Datos y exclusiones

Estado	Clasificación	Sincronización
Registros de <code>tasks.local.json</code>	Funcional e histórico	Sí, por <code>id</code>
<code>_sync</code> por registro	Origen, creación, modificación, revisión, tombstone y hash	Sí
<code>credentials.local.json</code>	Credencial local	Nunca
<code>notifications.local.json</code>	Token/configuración local	Nunca
<code>telegram.users.local</code>	Autorizaciones locales	Nunca
<code>telegram.offset.local</code> , <code>telegram.state.local</code>	Estado efímero	Nunca
<code>sync-state.local.json</code>	Base de comparación Nexus	No
<code>sync-audit.local.jsonl</code>	Auditoría sin valores privados	No
<code>backups/</code> , logs, PID, red y <code>app.local.env</code>	Backup/configuración/estado local	Nunca

La migración idempotente conservó los 18 registros y eliminó `username` y `password` del histórico. Las tareas consultan `credentials.local.json` únicamente al ejecutarse. `/api/settings` ya no devuelve la contraseña al navegador.

Runtime revalidado el 13/08/2026

Elemento	Evidencia
GitHub / Nexus	<code>main</code> · <code>6df1698d3346db5c35f7d173b5d7dc2567ce8a5e</code>
Contenedores	<code>reserva-pistas-nginx</code> y <code>reserva-pistas-app</code> activos
Publicación Nexus	Nginx <code>192.168.18.220:8083</code> → <code>80</code> ; backend sin puerto host
Persistencia	<code>/opt/data/reserva-pistas</code> → <code>/app/data</code> en lectura/escritura
Usuario de app	UID/GID <code>1000:1000</code>
Autoarranque	<code>restart: unless-stopped</code>
DigitalOcean	<code>reserva-pistas.service</code> activo como <code>reserva:reserva</code>
Producción	<code>/padel/</code> devolvió <code>401</code> sin credenciales, como se esperaba

Los hashes de `app.py` , `templates/index.html` y `sync_engine.py` desplegados en DigitalOcean coincidieron con `main` . Se ejecutaron **21 pruebas** locales sobre una copia limpia y todas terminaron correctamente. No se hicieron reservas, cancelaciones, sincronizaciones, notificaciones ni escrituras sobre datos reales.

Fusión y conflictos

El motor valida JSON e identificadores, calcula hashes canónicos por registro y compara cada lado con la base de la última sincronización:

1. incorpora registros presentes solo en el origen;
2. propaga cambios únicamente del origen;

3. conserva e informa cambios únicamente del destino;
4. conserva registros presentes solo en el destino;
5. propaga cancelaciones y tombstones explícitos;
6. si el mismo `id` cambió en ambos lados, no escribe hasta elegir Nexus, DigitalOcean o cancelar;
7. propaga el lado elegido por la persona;
8. una segunda simulación sin cambios no escribe ni crea un backup innecesario.

Los conflictos visibles se limitan a campos operativos no sensibles. Cada escritura usa bloqueo entre procesos, fichero temporal, `fsync`, reemplazo atómico, verificación de la huella simulada y backup previo.

Tareas activas

El destino no se modifica mientras tenga tareas `queued` o `running`. Si una tarea activa del origen se incorpora al otro servidor, la copia queda neutralizada como `cancelled` / `sync_neutralized`; conserva el histórico pero no inicia un segundo ejecutor. No existe sincronización automática ni periódica.

Panel administrativo

En **Ajustes** → **Sincronización administrativa** aparecen:

- DigitalOcean → Nexus ;
- Nexus → DigitalOcean ;
- estado del canal seguro;
- simulación con nuevos, actualizados, sin cambios, cancelaciones, conflictos, tareas activas y backup previsto;
- resolución humana por `id` ;
- confirmación explícita, protección contra doble clic y una sola operación concurrente;
- listado y restauración confirmada de backups.

En Nexus todo el backend usa Basic Auth local guardado fuera de Git. En DigitalOcean se conserva el Basic Auth de Nginx. Si falta conectividad o permisos, los botones quedan deshabilitados.

Backups y recuperación

Backups verificados:

Servidor	Backup	Resultado
DigitalOcean	<code>predeploy-20260810T014853+0200</code>	Datos, configuración, código, plantilla y unidad systemd; JSON y manifiesto SHA-256 válidos
Nexus	<code>tasks.local.json.20260810T015121+0200.backup</code>	Estado vacío previo a la primera escritura; JSON válido y restaurable

Restaurar desde el panel exige confirmación y crea antes otro backup del estado reemplazado. Backups, credenciales y logs nunca atraviesan el canal de sincronización.

Primera sincronización comprobada

Paso	Resultado real
DigitalOcean antes de migrar	18 registros: 12 cancelados, 6 reservados, 0 activos, 0 duplicados
Migración de secretos	Sin credenciales incrustadas; hash funcional anterior y posterior idéntico

Paso	Resultado real
Simulación DigitalOcean → Nexus	18 nuevos, 12 cancelaciones históricas, 0 conflictos, 0 activos
Simulación Nexus → DigitalOcean	18 solo en destino, 0 conflictos; producción no se escribió
Aplicación	Solo DigitalOcean → Nexus
Segunda simulación, ambas direcciones	18 sin cambios, 0 nuevos, 0 actualizados, 0 conflictos
Hash normalizado común	db9a806429479d9e83c83724ca67b5e072ca2ab29b94fbde9dbd19475342dc09
Configuración local	Credenciales y notificaciones de producción conservaron hash; Nexus no recibió secretos ni estado Telegram

Pruebas y entrega

- 21 pruebas: cambios unilaterales, altas en ambos lados, conflictos, cancelaciones, duplicados, repetición, interrupción, JSON inválido, conectividad, tareas activas, confirmación y restauración.
- Compilación Python y sintaxis JavaScript.
- CI con build Docker y ejecución real como UID/GID 1000.
- Nexus: build, Compose, HTTP 401/200, panel, persistencia, responsive CSS y logs sin errores.
- DigitalOcean: systemd, backend, Nginx, HTTP 401 público, migración equivalente y logs sin warnings.
- PR funcional [#4](#), corrección de runtime [#5](#) y guía validada [#6](#), todos fusionados mediante squash.
- El PR borrador #1 quedó cerrado al quedar incorporado y reconciliado en #4.

Operación

```
# Nexus
cd /opt/apps/Reserva-Pistas-UTP
git switch main
git pull --ff-only origin main
docker compose up -d --build

# Estado
curl -I http://192.168.18.220:8083/
docker compose ps

# Producción
systemctl is-active reserva-pistas nginx
nginx -t
journalctl -u reserva-pistas -n 100 --no-pager
```

No ejecutar reservas ni enviar notificaciones para probar el handover. Las comprobaciones de datos se realizan con simulación, hashes, recuentos e identificadores normalizados.

Aplicación · Consumos Cupra

Aplicación personal para registrar y analizar consumos de combustible. Fue creada inicialmente con AI Studio y remediada con Codex sin alterar su funcionalidad.

Estado actual

Campo	Valor
Repositorio	ratienza/Consumos_Cupra · privado
main desplegado	9f66a368a1dfd58e5b52741e263e77866397a7f7
Build	7db2eddf-8b54-46fa-a04b-1abcee75d72c
Imagen	cosnumos-cupra:9f66a368...
Digest	sha256:df935d3ace3ce6a319094dccfbae80606bc4a3c6d3044f87251d350443be8ed5
Runtime	Google Cloud Run · revisión cosnumos-cupra-00009-pon · tráfico 100 %
Rollback disponible	cosnumos-cupra-00007-b7b
Nexus / DigitalOcean	Sin runtime canónico; App Launch únicamente enlaza la aplicación

Arquitectura y despliegue

```
AI Studio / Codex → GitHub → Cloud Build → Artifact Registry → Cloud Run
```

React 19, Vite, Tailwind y Recharts forman el frontend. Express sirve el bundle y las APIs de registros, configuración, pendientes y diagnóstico. Google Sheets/Apps Script, Gemini y webhooks son dependencias externas opcionales; sus credenciales permanecen fuera de Git.

El trigger productivo construye desde GitHub y crea una revisión candidata con `--no-traffic`, label `candidate`, `entrypoint: gcloud` y `CLOUD_LOGGING_ONLY`. La promoción de tráfico es un paso controlado posterior.

Validación y operación

La revisión activa se validó con HTTP `200` en raíz, manifest, health, APIs GET no destructivas, configuración, pendientes, iconos, JavaScript y CSS. El build reproducible usa el lockfile y `npm ci`; lint, build e imagen Docker quedaron validados antes de la promoción.

El rollback consiste en devolver el tráfico a `cosnumos-cupra-00007-b7b` o desplegar de nuevo su imagen conocida. No requiere reconstruir el código ni intervenir DigitalOcean o Nexus.

Seguridad y deuda

- No mostrar valores de variables, secretos o integraciones externas.
- Los pushes a ramas conectadas pueden activar Cloud Build; los cambios productivos requieren revisar trigger, candidato y tráfico.

- `npm audit` registra cuatro vulnerabilidades conocidas: una baja, una moderada y dos altas. No se ejecutó `npm audit fix`; su tratamiento debe hacerse en un cambio separado y probado.

Aplicación · CV de Raúl

Currículum y portfolio profesional público creado con AI Studio, con frontend Vite/Tailwind y descarga de PDF generada durante la compilación.

Estado actual

Campo	Valor
Repositorio	ratienza/CV-Raul-IA-Estudio-Google- · privado
main	0da08cfa98e5ad9e5e81ed48ac4cd4428360d618
Producción	Firebase Hosting · proyecto replicant-lab
Versión activa observada	092bfc
Backend público identificado	cv-backend
URL	https://cv.raulatienza.com · HTTP 200
Nexus	Checkout limpio de consulta; sin contenedor, servicio o puerto

Arquitectura real

La cadena demostrada actualmente es:

```
despliegue manual Firebase Hosting → infraestructura Google/Firebase → cv.raulatienza.com
```

No está demostrada una cadena automática GitHub → producción ni la relación exacta entre el SHA actual y la versión Firebase 092bfc .

El servicio Cloud Run cv-raulatienza-com del proyecto consumos-cupra , región europe-west1 , conserva una revisión placeholder y un trigger fallido. Es residual y no constituye la producción pública. La reescritura Cloud Run declarada en firebase.json tampoco corresponde al servicio público observado.

Build, validación y rollback

El proyecto usa Vite, Tailwind, Node 20 y una generación PDF con PDFKit. El Dockerfile disponible construye con Node y sirve el resultado mediante Nginx, pero no describe el runtime canónico actual.

Producción, /health y /manifest.json respondieron 200 ; las dos últimas rutas son fallback de la SPA, no healthcheck ni manifest independientes. Firebase conserva versiones anteriores, por lo que existe capacidad de rollback desde Hosting.

Seguridad y deuda POST-CARTERA

No existe un problema crítico actual. Queda aceptado para después de Cartera:

- reconciliar Firebase, Cloud Build y configuraciones residuales;
- retirar o corregir el trigger obsoleto y demostrar trazabilidad SHA → versión Firebase;

- lockfile, reproducibilidad, CI, tests y typecheck;
- restos React, metadata Gemini, artefactos residuales, cabeceras, caché, protección de `main` y UX de revelado de contacto.

El checkout de Nexus no debe ejecutarse ni utilizarse para desplegar o restaurar producción.

Aplicación · Control de Red

Panel PowerShell para inventariar, nombrar y revisar dispositivos de la red local desde Replicant/Windows.

Estado auditado

Campo	Valor
Desarrollo	PowerShell + Codex
Repositorio	ratienza/control-red · privado
main	0e285d26f10ccb58e58d3ebbef35379b00a4b41d
Replicant	Herramienta local; no se ejecutó un escaneo durante la auditoría
Nexus	Checkout en /opt/apps/control-red ; sin contenedor, servicio o puerto
Entrada	ABRIR_PANEL.cmd → panel-control-red.ps1
Persistencia	Inventario JSON y snapshots versionados en el repositorio actual

Funcionalidad comprobada

El código implementa una interfaz PowerShell para descubrimiento, inventario, alias y snapshots de red. La sintaxis completa del script se analizó correctamente sin ejecutarlo, evitando escaneos o cambios sobre dispositivos.

Operación y rollback

La herramienta debe ejecutarse desde Replicant, donde existen PowerShell y acceso a la LAN. El checkout de Nexus es solo una copia de código/datos y no convierte el panel en aplicación web.

El rollback de código consiste en volver a un commit conocido mediante rama/PR. Los inventarios y snapshots no deben reemplazarse ni eliminarse automáticamente: requieren copia y revisión específica por contener estado del entorno.

Seguridad y pendientes

- El repositorio privado contiene inventario y snapshots reales versionados. Esto contradice el criterio global de mantener datos vivos fuera de Git, aunque su visibilidad sea privada.
- No se exponen aquí direcciones MAC, nombres de personas ni detalles del inventario.
- Pendiente: separar datos operativos del código, añadir ejemplos anonimizados y documentar backup/recuperación antes de cualquier limpieza.
- No está desplegado ni validado como servicio en Nexus.

Aplicación · Cartera Estratégica

Aplicación desarrollada con Codex y Python/Streamlit para operación local en Replicant.

Estado

Aplicación Python/Streamlit operativa como MVP local en Windows, con versión documental `v1.2.0` y SQLite privada fuera del repositorio. GitHub `main` estaba en `f2b319dea862469b90eb65fa20f8ae2d8c96875d` ; el checkout de trabajo observado en Replicant estaba atrasado en `3be6254` y no se modificó.

Entorno comprobado

El entorno vigente es Replicant/Windows con ejecución local de Streamlit. No existe en la documentación actual de la aplicación una decisión aprobada que convierta Nexus o Docker/Linux en su siguiente destino.

Consideraciones

- La base privada no debe entrar en Git.
- Tokens como EODHD deben inyectarse como secreto/entorno.
- Los datos de inversión pueden permanecer solo en local.
- Cualquier soporte Docker/Linux o despliegue remoto deberá decidirse e implementarse en el repositorio de la aplicación mediante rama y PR.

Arquitectura y funciones verificadas

El núcleo usa Python, SQLite y migraciones versionadas; Streamlit es la presentación. El repositorio incluye importación y conciliación, transacciones, FIFO, cash, backups, exportaciones, datos de mercado EODHD, estrategia 7A e inteligencia SMC 7B. Las credenciales se inyectan por entorno y la base, los backups, XLSX y exportaciones reales quedan fuera de Git.

La actualización debe hacerse únicamente sobre una copia y con backup SQLite verificado antes de migrar. El rollback usa el servicio de restauración y una copia probada; nunca se sustituye la base privada por un fichero del repositorio.

Pruebas del 13/08/2026

- `pytest --collect-only` : 369 pruebas recogidas.
- La suite aislada avanzó aproximadamente hasta el 58 % sin fallos antes del límite de cinco minutos.
- No se declara la suite completa como validada.
- No se abrió, copió, migró ni modificó la SQLite privada.

Pendiente

No está desplegada ni validada en Nexus. Tampoco debe presentarse Nexus como objetivo acordado hasta que exista una decisión versionada en el repositorio de la aplicación.

Aplicación · Replicant Lab

Documentación canónica del laboratorio: arquitectura, hosts, red, despliegues, operación, decisiones, pendientes y fichas técnicas de aplicaciones.

Estado auditado

Campo	Valor
Desarrollo	Codex / MkDocs / Mermaid
Repositorio	ratienza/infra-replicant-lab · main
Punto de partida del cierre	0134f5bdf932128de47545a03acab593027f797f
Nexus	/opt/apps/infra-replicant-lab · checkout limpio observado
Servicio	Compose docs · contenedor infra-replicant-docs
Imagen	infra-replicant-docs:local , construida desde el Dockerfile
URL / puerto	http://192.168.18.220:8082 · 192.168.18.220:8082 → 80/tcp
Reinicio	restart: unless-stopped

Arquitectura y funcionamiento

Markdown, mkdocs.yml y los recursos de docs/ son la fuente de verdad. El pipeline calcula una huella SHA-256, construye MkDocs en modo estricto y genera el dossier global y las fichas HTML/PDF individuales.

El Dockerfile usa dos etapas: Python/MkDocs construye el sitio y Nginx 1.27.4-alpine sirve únicamente el resultado estático. El runtime no usa mkdocs serve , bind mounts, base de datos ni almacenamiento persistente.

Actualización y rollback

```
cd /opt/apps/infra-replicant-lab
git switch main
git pull --ff-only origin main
docker compose up -d --build
```

La actualización se realiza solo después de integrar el PR. Para rollback se revierte el cambio mediante GitHub, se actualiza de nuevo main y se reconstruye exclusivamente este servicio; no se recupera código desde el contenedor.

Pruebas realizadas

- python scripts/docs_pipeline.py generate --screenshots : correcto.
- python scripts/docs_pipeline.py check : sincronía correcta.
- 32 páginas fuente y seis diagramas Mermaid validados por el pipeline.
- Dossier global de 52 páginas y ocho fichas individuales HTML/PDF sincronizadas.
- Runtime Nexus observado con respuesta 200 el 13/08/2026; la nueva versión se valida de nuevo tras el merge.

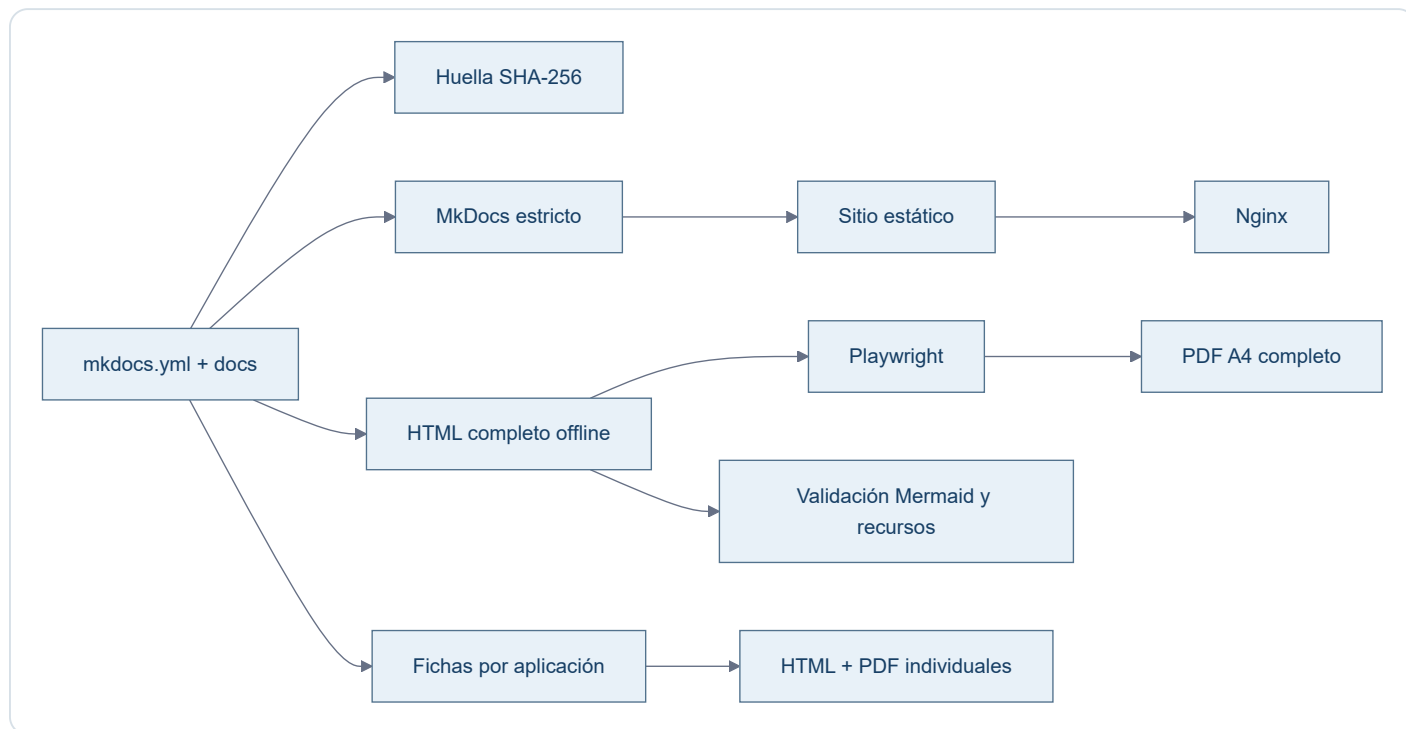
Seguridad, dependencias y pendientes

- No almacena secretos, datos vivos ni backups; GitHub contiene solo documentación y configuración versionable.
- La tipografía remota del tema es opcional: una caída de Google Fonts no bloquea el contenido ni los recursos locales.
- Depende de GitHub para actualización y de Docker para reconstrucción; no depende de las aplicaciones documentadas para servir el sitio.
- Pendiente transversal: definir y probar la política de backup/restauración de Nexus. La documentación está en Git, pero eso no sustituye los backups de datos de otras aplicaciones.

Autodocumentación

Estado implementado

La documentación mantiene una fuente humana canónica (`mkdocs.yml` , `docs/` y recursos) y una cadena reproducible para construir el sitio y sus artefactos derivados.



Herramientas versionadas

- `scripts/docs_pipeline.py` : orquesta construcción, generación, sincronía y validaciones estructurales.
- `scripts/render_portables.mjs` : renderiza Mermaid en Chromium y produce PDF y evidencias.
- `scripts/render_app_portable.mjs` : valida escritorio/móvil y genera cada PDF individual.
- `scripts/portable.css` : presentación compartida por HTML offline y PDF.
- `requirements-docs.txt` : dependencias Python exactas.
- `package.json` y `pnpm-lock.yaml` : Mermaid, Playwright y árbol Node fijados.
- `docs/javascripts/mermaid.min.js` : runtime derivado de la versión Mermaid fijada y comprobado contra `node_modules` .

Huella y reproducibilidad

La huella incluye configuración MkDocs, páginas en `nav` , recursos y herramientas que afectan a los portables. Los HTML deben coincidir byte a byte con la salida esperada. Los PDF se comprueban mediante esa huella visible, cobertura textual y equivalencia de paginación con una regeneración de control.

Temporales

Todos los resultados intermedios viven bajo `.build/docs-pipeline/` . Entornos virtuales, `node_modules` , navegadores, cachés, capturas y reportes no entran en Git.

Flujo de actualización

1. modificar exclusivamente fuentes MkDocs;
2. ejecutar `python scripts/docs_pipeline.py generate ;`
3. revisar el sitio, HTML, PDF y capturas;
4. ejecutar `python scripts/docs_pipeline.py check ;`
5. revisar el diff completo;
6. commit, push y Pull Request;
7. tras merge, reconstruir el servicio estático en Nexus mediante Compose;
8. validar publicación y descargas en Nexus como un estado separado.

Decisiones arquitectónicas

Registro corto de decisiones que no conviene redescubrir.

Decisión	Motivo
Nexus usa IP fija <code>.220</code>	Identidad estable del servidor local
Replicant usa <code>.200</code>	Identidad estable del host físico
Aliases locales sin servidor DNS	Replicant resuelve <code>nexus</code> → <code>192.168.18.220</code> y <code>replicant</code> → <code>192.168.18.200</code> mediante <code>hosts</code> ; no se añade infraestructura DNS
Docker como estándar	Mantener limpio el host Ubuntu
GitHub como source of truth	Reproducibilidad e histórico
Datos y secretos fuera de Git	Seguridad y separación de responsabilidades
UFW restringe SSH a LAN	Reducir superficie de exposición
No usar Portainer/Webmin/Cockpit por defecto	Evitar complejidad innecesaria
Ramas por cambio lógico	Facilitar revisión y mantener <code>main</code> estable
MkDocs es la fuente documental canónica	<code>mkdocs.yml</code> , <code>docs/</code> y sus recursos definen contenido, estructura y presentación
HTML/PDF son artefactos derivados	Deben sincronizarse mediante un proceso reproducible; nunca prevalecen sobre MkDocs
Nexus sirve una imagen estática Nginx	<code>Dockerfile</code> , Compose y CI comparten build MkDocs estricto; el despliegue requiere reconstrucción
Validación por entorno	Git, prueba local, Nexus y producción son estados distintos y no se extrapolan
Dependencias documentales fijadas	Python/Node, MkDocs, Mermaid y Playwright se resuelven desde lockfiles versionados
Huella SHA-256 para portables	Evita timestamps variables y permite demostrar sincronía inequívoca con las fuentes
PDF tratado como binario	<code>.gitattributes</code> anula <code>diff=astextplain</code> y conversiones CRLF de Git para Windows
Rutas portables únicas	HTML y PDF viven exclusivamente en <code>docs/downloads/</code>
App Launch usa puerto <code>80</code> en Nexus	Permite <code>http://nexus/</code> sin puerto explícito; <code>8080</code> queda libre
App Launch selecciona catálogo al desplegar	Una presentación común y un único <code>apps.json</code> activo por host evitan filtrar enlaces internos
Checkouts no equivalen a servicios	CV y Control de Red pueden existir en disco de Nexus sin declararse desplegados
Fichas técnicas derivadas por aplicación	Cada aplicación tiene HTML/PDF generado desde su Markdown canónico
Salones AV liga <code>8081</code> a la IP LAN	<code>192.168.18.220:8081:80</code> evita publicar el servicio en todas las interfaces y queda versionado desde <code>8c0bc08</code>
App Launch es navegación, no runtime	Una tarjeta puede apuntar a servicios locales o remotos sin trasladar su ejecución al host del launchpad
Consumos Cupra usa Cloud Run	La cadena canónica es GitHub → Cloud Build → Artifact Registry → Cloud Run; DigitalOcean y Nexus no son su producción

Decisión	Motivo
CV usa Firebase Hosting	La producción observada pertenece al proyecto replicant-1ab ; el Cloud Run placeholder no es producción

Change Log

Acceso estable al histórico diario de Raul Lab.

- [13 de agosto de 2026](#)
- [9 de agosto de 2026](#)
- [8 de agosto de 2026](#)

Las entradas se ordenan de la fecha más reciente a la más antigua.

Change Log · 13 de agosto de 2026

Auditoría global y App Launch

- Confirmada la implementación multientorno de App Launch en `ced6e14`.
- Validados DigitalOcean por HTTP/HTTPS y Nexus por el puerto `80`.
- Confirmado `8080` libre y `8081-8083` sanos.
- Verificada la separación exacta de catálogos y la ausencia de referencias Nexus en DigitalOcean.

Inventario de aplicaciones

- Auditados repositorio, SHA, arquitectura, despliegue y estado real de ocho aplicaciones.
- Incorporadas fichas individuales para App Launch, Consumos Cupra, CV y Control de Red.
- Ampliadas las fichas de Salones AV, Reserva-Pistas-UTP y Cartera Estratégica.
- Incorporada una ficha individual de Replicant Lab, separada del dossier global.
- Diferenciados explícitamente servicios, checkouts de solo lectura, producción cloud y MVP local.

Evidencias y pendientes

- Reservas: 21 pruebas correctas.
- Consumos: TypeScript y build correctos.
- CV: build y PDF correctos.
- Salones: enlaces locales y rutas publicadas correctos.
- Control de Red: sintaxis PowerShell correcta sin escaneo.
- Cartera: 369 pruebas recogidas; ejecución parcial sin fallos hasta el límite temporal, no declarada como suite completa.
- Conservadas como pendientes la deriva Compose de Salones, la trazabilidad de despliegue de Consumos, la configuración Cloud del CV, los datos versionados de Control de Red y la política global de backups.

No se modificaron CV/Firebase/Cloud Run, Cartera, red, firewall, DNS, certificados, secretos ni datos privados.

Fase 2A · Remediación de Salones AV

- Resuelta la deriva de `compose.yml` mediante el PR `salones-av-valencia-palace#2`.
- Versionado el bind LAN `192.168.18.220:8081:80` en `main` (`8c0bc08`).
- Reconciliado Nexus preservando y comparando el cambio local antes del avance rápido.
- Verificados Compose, blob Git, checkout limpio, bind efectivo y las siete rutas con respuesta `200`.
- No se modificaron contenido AV, fuentes documentales, datos, otros contenedores ni producción DigitalOcean.

Consolidación final pre-Cartera

- Cerradas documentalmente las fases 2A, 2B y 2C sin reabrir remediaciones.
- Consumos Cupra queda trazado desde `9f66a368` hasta Cloud Run `cosnumos-cupra-00009-pon`; DigitalOcean y Nexus no son su runtime.
- El CV queda identificado en Firebase Hosting del proyecto `replicant-lab`, operativo en `cv.raulatienza.com`, con deuda aceptada POST-CARTERA.
- App Launch queda definido como catálogo y navegación: una tarjeta no demuestra runtime local.
- La navegación histórica se reduce a hitos y las pruebas detalladas permanecen en las fichas de las ocho aplicaciones.

9 de agosto de 2026

Change Log diario de Raul Lab. Reúne los cambios documentales, técnicos y operativos de la fecha, del hito más reciente al más antiguo.

Cierre integral de Reserva-Pistas-UTP · validación final 10/08/2026

- Los [PR #4](#), [#5](#) y [#6](#) integraron mediante squash el motor de sincronización, la corrección del usuario de runtime y la guía de despliegue validada. El borrador #1 quedó cerrado al ser sustituido.
- La aplicación dejó de exponer contraseñas y de incluir credenciales en las tareas. El almacenamiento incorpora validación estricta, IDs estables, bloqueo entre procesos, escritura atómica, copias, restauración, ledger y detección de conflictos.
- El panel permite vista previa y confirmación explícita en ambas direcciones, evita dobles ejecuciones y presenta conflictos solo con campos no sensibles. La transferencia se realiza desde Nexus mediante clave dedicada y comando SSH forzado en DigitalOcean.
- Se superaron 21 pruebas, compilación Python, sintaxis JavaScript, configuración y construcción Docker, controles HTTP y revisión de logs.
- Antes de intervenir se conservaron copias de seguridad; los datos funcionales y los ficheros locales de credenciales y notificaciones de producción mantuvieron su integridad.
- Se replicaron de DigitalOcean a Nexus 18 registros —12 cancelados y 6 reservados—, sin tareas activas, duplicados ni conflictos. Después, ambas vistas previas convergieron en 18 elementos sin cambios y el documento normalizado compartió el hash `db9a806429479d9e83c83724ca67b5e072ca2ab29b94fbde9dbd19475342dc09`.
- Nexus quedó operativo en `192.168.18.220:8083` y DigitalOcean en `https://app.raulatienza.com/padel/`; no se alteraron DNS, certificados, puertos ni la topología pública. El [issue #2](#) quedó cerrado con evidencias.

Corrección final de experiencia documental · PR #11

- El [PR #11](#) integra la corrección final de cronología, visión transversal de repositorios y experiencia portable.
- La sección **Cambios** pasa a presentarse como **Change Log**, con una cronología única, fechas visibles y contexto técnico y operativo.
- **Pendientes** amplía su alcance a los ocho repositorios vinculados a Raul Lab comprobados en GitHub; `ratienza/python` queda excluido expresamente.
- El HTML independiente recupera una experiencia multipágina dentro de un único fichero autocontenido: índice persistente, una vista documental activa, anterior/siguiente, fragmentos directos, historial atrás/adelante, búsqueda local y adaptación móvil.
- El PDF mantiene su modelo editorial completo; HTML y PDF continúan generándose desde la fuente MkDocs canónica y no desde exportaciones antiguas.

18:55 · Estado real de Nexus registrado · PR #10

- El [PR #10](#) integró la evidencia de despliegue y cerró los estados que todavía figuraban como pendientes.
- `main` quedó en `cd09dec4cb083f7a761e09648e23404cfd35da0c`.
- Nexus quedó sincronizado, con Nginx estático estable en `192.168.18.220:8082`, cinco Mermaid y descargas verificadas.

18:39 · Pipeline reproducible y runtime estático · PR #9

- El [PR #9](#) integró el pipeline reproducible de documentación.
- MkDocs — `mkdocs.yml`, `docs/` y sus recursos— quedó reafirmado como fuente canónica de contenido, estructura y presentación.
- Docker, Compose y CI convergieron en `mkdocs build --strict` y una imagen final Nginx; desaparecieron `mkdocs serve`, los bind mounts y el runtime de desarrollo en Nexus.
- Mermaid 11.16.1 pasó a servirse localmente, sin CDN, y los cinco diagramas quedaron disponibles también offline.
- Se generaron un HTML autocontenido completo y un PDF A4 completo desde las 27 páginas de navegación.
- Las rutas canónicas quedaron fijadas en `docs/downloads/Replicant-Lab.html` y `docs/downloads/Replicant-Lab.pdf`; se eliminó el HTML duplicado obsoleto.

- La validación local incluyó MkDocs estricto, enlaces, recursos, Docker, escritorio, móvil, HTML `file://` , PDF y ausencia de CDN, localhost, WebSocket y recarga en vivo.
- El squash resultante fue `f457c57b472515afe46025078c7342dd5a75a7f1` .

Despliegue y validación real en Nexus

- Se actualizó exclusivamente `/opt/apps/infra-replicant-lab` mediante avance rápido y `docker compose up -d --build` .
- El contenedor `infra-replicant-docs` quedó activo con Nginx `1.27.4` , reinicios `0` y publicación `192.168.18.220:8082` → `80` .
- Se comprobaron navegación, recursos, presentación de escritorio y móvil, cinco Mermaid, HTML offline y PDF completo.
- Las descargas reales coincidieron byte a byte con Git:
- HTML: `17a2746d2e11e26d0302a57633c5b1b05119348d8e4a4629f00d67ab8cfce6a1` .
- PDF: `028a135d7645912569c5d6c6c13b7c885b250084e3b269add7d10f89f627e13b` .
- Los logs finales no mostraron errores, `404` , recarga en vivo, WebSocket ni dependencias esenciales externas.

17:11 · Reconciliación y consolidación documental · PR #8

- El [PR #8](#) reconcilió las fuentes MkDocs con el estado comprobado de GitHub y Nexus.
- Se separaron con claridad Replicant/Hyper-V, Nexus, DigitalOcean y los repositorios propios de cada aplicación.
- README, navegación, operación, despliegue, aplicaciones y pendientes quedaron alineados; el procedimiento de Nexus distinguió primer arranque de actualización ordinaria.
- El squash resultante fue `9c31728bfb0a4399dc736bea59f7ed50ed480bf3` .

16:07 · Contrato operativo raíz · PR #7

- El [PR #7](#) incorporó `AGENTS.md` .
- El contrato fijó la jerarquía entre verdad técnica versionada, documentación MkDocs canónica y artefactos HTML/PDF derivados.
- También formalizó separación de entornos, protección de datos, flujo rama–pruebas–PR–merge y prohibición de desplegar o modificar producción sin autorización.
- El squash resultante fue `01d859fac7b7907ae92cf65bda2ad50cfb0e738f` .

8 de agosto de 2026

Change Log diario de Raul Lab. Se conserva íntegramente el histórico previamente registrado para esta fecha.

- Replicant queda con IP fija `192.168.18.200`.
- Nexus queda con IP fija `192.168.18.220` mediante Netplan.
- SSH por clave consolidado.
- UFW activado con SSH permitido solo desde la LAN.
- Se verifican Docker, Git y actualizaciones automáticas.
- Salones AV queda accesible desde la LAN en `192.168.18.220:8081`.
- Replicant Lab queda publicado en Nexus por `192.168.18.220:8082`.
- Reserva-Pistas-UTP pasa de un staging manual a un despliegue Docker Compose completo en Nexus: backend Python + Nginx en red privada Docker, acceso LAN por `192.168.18.220:8083`.
- El backend de Reserva-Pistas se hace configurable mediante `APP_HOST`, `APP_PORT` y `APP_DATA_DIR`, manteniendo valores por defecto compatibles con el despliegue anterior.
- La persistencia de Reserva-Pistas se separa en `/opt/data/reserva-pistas` y se monta como `/app/data`.
- El backend Docker se ejecuta como UID/GID `1000:1000` y ambos servicios usan `restart: unless-stopped`.
- Se valida HTTP `200`, persistencia tras recreación de contenedores y recuperación automática tras reinicio completo de Nexus.
- UFW permite `8083/tcp` exclusivamente desde `192.168.18.0/24`.
- Se formaliza la configuración Nexus en `ratienza/Reserva-Pistas-UTP` y se integra en `main` mediante PR #3.
- DigitalOcean permanece sin cambios y continúa como producción de Reserva-Pistas-UTP.
- Queda pendiente sincronizar a Nexus el histórico/estado vigente de `tasks.local.json` desde DigitalOcean, verificando antes que no existan tareas activas que puedan duplicarse.
- La portada documental incorpora accesos de descarga al HTML autónomo y al PDF de Replicant Lab.
- DNS local y backups se mantienen pendientes conscientemente.
- Se consolida `infra-replicant-lab` como documentación viva.